

Final Test

Predicting Product Ratings Using Natural Language Processing

Hongjun Kim, Thien Phung, Randy Van Waes, and Sophia Choi

Introduction

In this project, our group is looking to understand how textual reviews relate to star ratings. Using only the text of the Amazon product reviews, we aim to estimate the numeric star score that the user actually assigned.

In order to do this, we:

1. Treat the original 1–5 star rating as our numeric outcome to predict from text.
2. Reconstruct each review into a cleaned string and compute TF–IDF feature matrices for both unigrams and bigrams.
3. Align the `'rating'` vector to the rows of those matrices.
4. Fit regression models on each feature set, using k-fold cross-validation to select the best penalty.
5. Report training vs. test RMSE across those folds and visualize the RMSE curves to assess model stability.

Data Description

We obtain our raw Amazon product review dataset by downloading from Kaggle. This dataset includes 1465 observations of Amazon sales transactions with 16 column variables, including 12 string variables, 2 integer variables, and 2 URLs.

These columns include:

product_id - Product ID
product_name - Name of the Product
category - Category of the Product
discounted_price - Discounted Price of the Product
actual_price - Actual Price of the Product
discount_percentage - Percentage of Discount for the Product
rating - Rating of the Product
rating_count - Number of people who voted for the Amazon rating
about_product - Description about the Product
user_id - ID of the user who wrote review for the Product
user_name - Name of the user who wrote review for the Product
review_id - ID of the user review
review_title - Short review
review_content - Long review
img_link - Image Link of the Product
product_link - Official Website Link of the Product

Because our focus is on predicting the actual 1–5 star rating from review text, we use only three columns as our main model inputs:

review_content: the raw review text (our predictor)

rating: the 1–5 star rating (which we convert to positive vs. negative)

review_id: a unique identifier for each review

Libraries and Setup

First, load the packages needed for data wrangling, text preprocessing, and modeling. This matches the style of our previous PSTAT 134 labs and assignments

```
library(dplyr)
library(vroom)
library(readr)
library(tidyr)
library(tidytext)
library(stringr)
library(tm)
library(glmnet)
library(tidyverse)
library(reshape2)
library(wordcloud)
library(ggraph)
library(httr)
library(igraph)
library(data.table)
library(textdata)
library(ggplot2)
library(ggrepel)
library(plotly)
library(umap)
library(word2vec)
library(tm)
library(kableExtra)
library(readr)
library(proxy)
library(rsample)
data("stop_words")
```

```
#Import data
amazon <- read_csv("amazon.csv")
```

```
## Warning: One or more parsing issues, call `problems()` on your data frame for details,
## e.g.:
##   dat <- vroom(...)
##   problems(dat)
```

```
## Rows: 1465 Columns: 16
## — Column specification —————
## Delimiter: ","
## chr (14): product_id, product_name, category, discounted_price, actual_price...
## dbl (1): rating
## num (1): rating_count
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
head(amazon,5)
```

```
## # A tibble: 5 × 16
##   product_id product_name          category discounted_price actual_price
##   <chr>      <chr>              <chr>      <chr>          <chr>
## 1 B07JW9H4J1 Wayona Nylon Braided USB to... Compute... ₹399          ₹1,099
## 2 B098NS6PVG Ambrane Unbreakable 60W / 3... Compute... ₹199          ₹349
## 3 B096MSW6CT Sounce Fast Phone Charging ... Compute... ₹199          ₹1,899
## 4 B08HDJ86NZ boAt Deuce USB 300 2 in 1 T... Compute... ₹329          ₹699
## 5 B08CF3B7N1 Portronics Konnect L 1.2M F... Compute... ₹154          ₹399
## # i 11 more variables: discount_percentage <chr>, rating <dbl>,
## #   rating_count <dbl>, about_product <chr>, user_id <chr>, user_name <chr>,
## #   review_id <chr>, review_title <chr>, review_content <chr>, img_link <chr>,
## #   product_link <chr>
```

Data Cleaning

We first begin by cleaning the raw dataset. We scan our two modeling inputs (**review_content**, **rating**) for NA. Any rows containing missing values are dropped. Then, to avoid counting the same review twice, we remove any duplicate rows.

To clean each review's text column, we use the **clean_text()** function.

With this function we:

1. Remove all URLs, non-ASCII characters, and apostrophes
2. Split camel case words into their component parts
3. Replace remaining punctuation with spaces
4. Convert everything to lowercase
5. Collapse repeated whitespace into single spaces
6. Trim leading/trailing blanks

After all this, we overwrite the original **review_content** column with our fully normalized text. We also split the **category** column into a list of individual category labels under **category_list**.

```
library(dplyr)
library(stringr)
library(tidytext)
library(tidyr)
library(tm)
```

#Clean Data

```
rows_with_missing <- amazon %>% #Check for missing values
  filter(if_any(everything(), is.na))
print(paste(nrow(rows_with_missing), " missing values")) #0 missing values so no further removal
needeed
```

```
## [1] "3 missing values"
```

```
amazon <- amazon %>% distinct() #Remove duplicates
```

```
pattern_to_remove <- "[[:punct:]]" #Remove any unnecessary characters
```

```
amazon$review_content <- amazon$review_content %>%
```

```
  tolower() %>%
```

#Convert to Lowercase

```
  str_replace_all("<[^>]+>", " ") %>%
```

#Remove any HTML elements

```
  str_remove_all("https?://[^\s]+") %>%
```

#Remove links starting with http or htt

ps

```
  str_remove_all("www\\.[^\s]+") %>%
```

#Remove links starting with www

```
  str_remove_all("'") %>%
```

#Remove apostrophes

```
  str_replace_all("([a-z])([A-Z])", "\\1 \\2") %>%
```

#Separate camelCase words

```
  str_remove_all("[0-9]+") %>%
```

#Remove numbers

```
  str_replace_all("[^[:ascii:]]", " ") %>%
```

#Remove emojis and non-ASCII characters

```
  str_replace_all(pattern_to_remove, " ") %>%
```

#Replace punctuation with space

```
  str_replace_all("\\s+", " ")
```

```
amazon <- amazon %>% #Seperate categories into array
```

```
  mutate(category_list = strsplit(category, "\\|"))
```

```
amazon$review_idmain <- amazon$review_id
```

#Tokenize data

```
reviews_token <- amazon %>%
```

```
  select(review_content) %>%
```

```
  rename(text = review_content) %>%
```

```
  unnest_tokens(word, text) %>%
```

```
  anti_join(stop_words) %>%
```

```
  count(word, sort=TRUE)
```

```
head(reviews_token, 30)
```

```
## # A tibble: 30 × 2
##   word      n
##   <chr>   <int>
## 1 product 3932
## 2 quality 2422
## 3 price   1395
## 4 cable   1380
## 5 phone   1076
## 6 charging 1008
## 7 nice     951
## 8 easy     945
## 9 battery  879
## 10 time    849
## # i 20 more rows
```

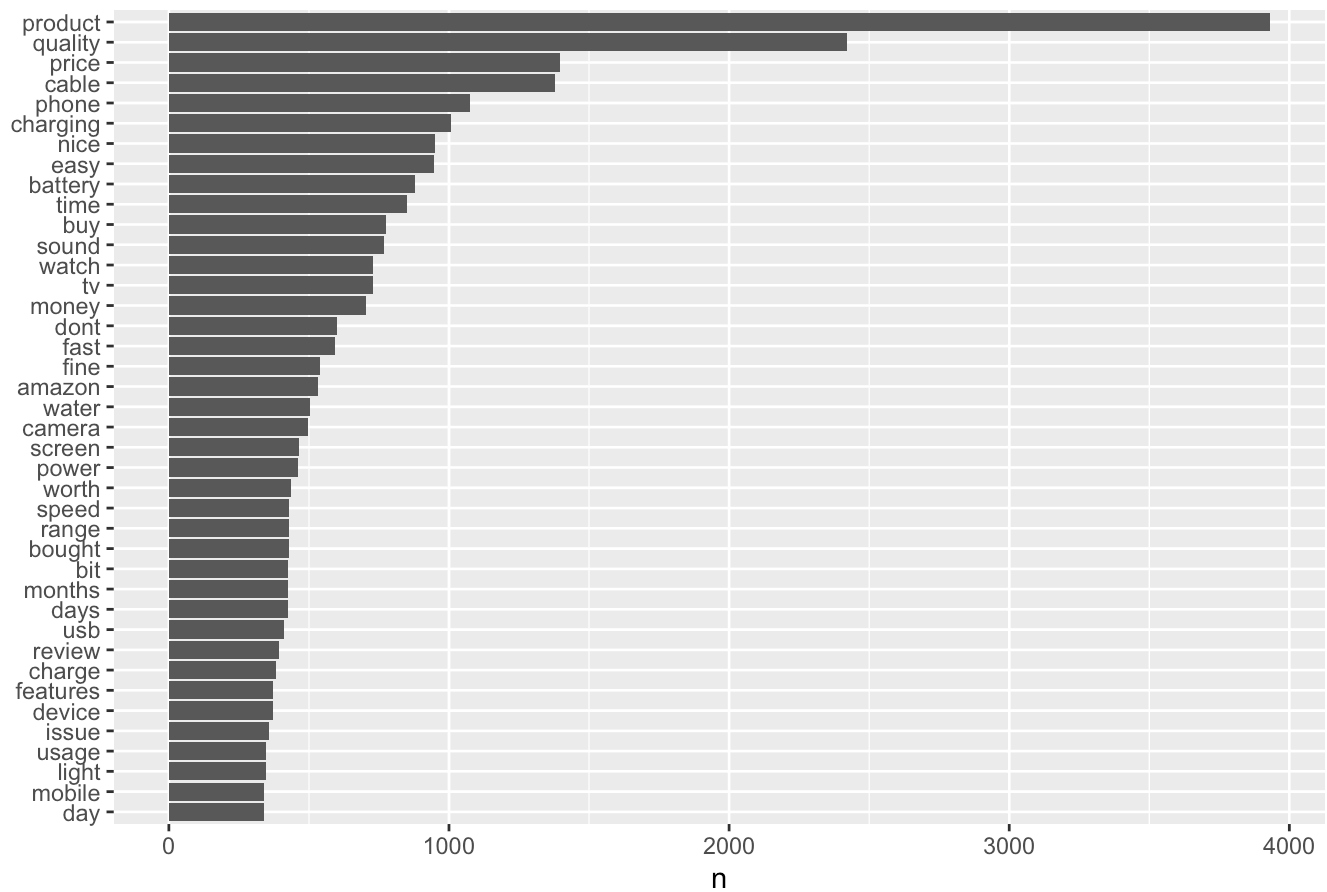
Methods/Results

Tokenized Data

As an initial step, we explore and chart the highest and mid-level word counts to identify which common and moderately common tokens might drive our sentiment models

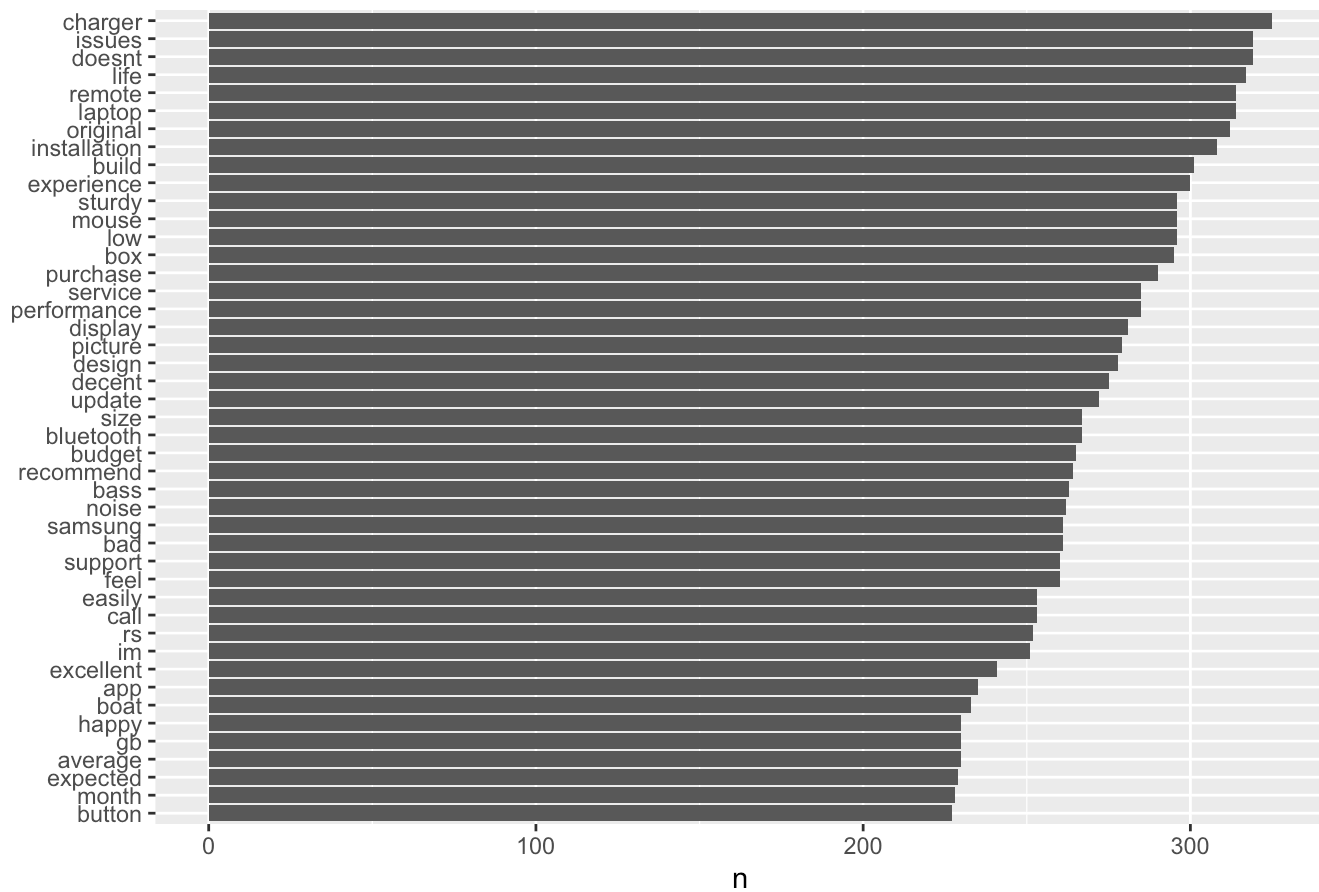
```
#Graphical representation of tokenized data
reviews_token %>%
  filter(n > 333) %>%
  mutate(word = reorder(word, n)) %>%
  ggplot(aes(n, word)) +
  geom_col() +
  labs(title = "Top Tokens (n > 333)",
       y = NULL)
```

Top Tokens (n > 333)



```
reviews_token %>%
  filter(225 < n) %>%
  filter(n < 333) %>%
  mutate(word = reorder(word, n)) %>%
  ggplot(aes(n, word)) +
  geom_col() +
  labs(title = "Top Tokens (n < 333)",
       y = NULL)
```

Top Tokens (n < 333)



```
reviews_token %>%
  count(word,n) %>%
  with(wordcloud(word, n, max.words = 100, scale=c(4,.5),colors=brewer.pal(8,"Dark2")))
```

```
## Storing counts in `nn`, as `n` already present in input
## i Use `name = "new_name"` to pick a new name.
```



Outputs

The first output, titled “Top Tokens (n > 333),” demonstrates that highly generic terms such as “product,” “quality,” and “price” dominate the reviews. The term “product” is the top term, appearing almost 4,000 times.

The second output, titled “Top Tokens (n < 333),” displays the mid-frequency terms such as “charger” and “issues” that carry specific sentiment cues, offering more targeted insights.

Sentiment Tables

Next, to identify tokens that carry positive or negative connotations, we merge our token frequency table with the NRC and Bing sentiment dictionaries.


```

#Get sentiments
nrc_negative <- get_sentiments("nrc") %>%
  filter(sentiment == "negative")
nrc_positive <- get_sentiments("nrc") %>%
  filter(sentiment == "positive")

#Get words within reviews
negative<- reviews_token %>%
  inner_join(nrc_negative, by = "word")
positive<- reviews_token %>%
  inner_join(nrc_positive, by = "word")

#Get total number of positive/negative sentiments
total_negative <- sum(negative$n)
total_positive <- sum(positive$n)

#Negative word count
negative%>%
  head(100) %>%
  kbl(caption = paste("Negative words; Total words:", total_negative, ")) %>%
  scroll_box(width = "400px", height = "500px")

```

Negative words; Total
words: 7158)

word	nsentiment
battery	879negative
noise	262negative
bad	261negative
weight	224negative
cheap	168negative
ram	123negative
calls	100negative
dust	98negative
remove	92negative
lag	88negative
missing	82negative
bottom	78negative
lower	78negative
wear	73negative
disappointed	70negative
fits	67negative
damage	66negative
useless	65negative
defective	63negative
rating	61negative

```
#Positive word count
positive %>%
  head(100) %>%
  kbl(caption = paste("Positive words; Total words:", total_positive, ")) %>%
  scroll_box(width = "400px", height = "500px")
```

Positive words; Total
words: 14773)

word	nsentiment
money	705positive
worth	435positive
charger	325positive
build	301positive
sturdy	296positive
decent	275positive
recommend	264positive
excellent	241positive
happy	230positive
weight	224positive
warranty	218positive
feature	214positive
clean	209positive
job	206positive
option	206positive
found	205positive
perfect	201positive
received	201positive
durable	194positive
satisfied	187positive

```
reviews_token %>%
  inner_join(get_sentiments("bing")) %>%
  count(word, n, sentiment, sort = TRUE) %>%
  acast(word ~ sentiment, value.var = "n", fill = 0) %>%
  comparison.cloud(colors = c("red", "blue"),
    max.words = 100)
```

[illegible]

The “Negative words” list shows the most frequent negative tokens. The top negative words such as “battery,” “noise,” and “bad” reflect common performance and quality issues.

Reconstruction and TF-IDF

11/24

```

# Load and clean Amazon reviews by tokenizing and removing unwanted words
clean_reviews <- amazon %>%
  mutate(review_id = row_number()) %>%                                # Assign a unique ID to each review
  unnest_tokens(word, review_content) %>%                             # Tokenize the reviews into individual words
  anti_join(stop_words, by = "word") %>%                             # Remove common stop words (like "the", "is", etc.)
  filter(!str_detect(word, "\\d")) %>%                                # Remove words that contain numbers
  filter(!word %in% stopwords("en")) %>%                             # Remove stop words using the 'stopwords' package

# Reconstruct cleaned reviews and merge them with their original ratings
final_reviews <- clean_reviews %>%
  group_by(review_id) %>%
  summarise(review_text = paste(word, collapse = " ")) %>%           # Rebuild each review as a cleaned sentence
  ungroup() %>%
  left_join(
    clean_reviews %>%
      select(review_id, rating) %>%                                # Get the original ratings for each review
    distinct(),
    by = "review_id"
  ) %>%
  filter(!is.na(rating)) %>%                                         # Remove any reviews that don't have a rating

```

Unigrams

We then compute TF-IDF at the one-word phrase level.

```

# Create Unigrams and Compute TF-IDF

text_df <- final_reviews %>%
  unnest_tokens(word, review_text) %>%                                # Tokenize the cleaned review text into individual words (unigrams)
  count(review_id, word) %>%                                         # Count the number of times each word appears in each review
  bind_tf_idf(word, review_id, n) %>%                                # Calculate Term Frequency-Inverse Document Frequency (TF-IDF)

# TF: importance of word in the specific review
# IDF: how unique the word is across all reviews

# View the First Few Rows of the Final Cleaned Data and TF-IDF Table

head(text_df) %>%                                                    # Preview the cleaned reviews with original ratings

```

```
## # A tibble: 6 × 3
##   review_id review_text                                rating
##   <int> <chr>                                <dbl>
## 1         1 durable charging fine toono complains charging fast product ...    4.2
## 2         2 cable connect phone android auto car cable strong connection...     4
## 3         3 durable sturdy nice product nice product                                3.9
## 4         4 product wire charges nice bought cable rs worthy product pri...    4.2
## 5         5 bought original apple rs fast apple charger option cheap pro...    4.2
## 6         6 product item strong usb cablevalue moneythanks amazon produc...    3.9
```

```
head(text_df, 30)                                # View the top 30 rows of the TF-IDF dataframe
```

```
## # A tibble: 30 × 6
##   review_id word          n    tf   idf tf_idf
##   <int> <chr>    <int> <dbl> <dbl> <dbl>
## 1         1 cable          2 0.0714  1.40 0.0998
## 2         1 charging       3 0.107   1.46 0.156
## 3         1 company        1 0.0357  2.46 0.0879
## 4         1 complains      1 0.0357  4.65 0.166
## 5         1 date           1 0.0357  3.23 0.115
## 6         1 durable        1 0.0357  2.28 0.0814
## 7         1 fast           1 0.0357  1.44 0.0515
## 8         1 fine           1 0.0357  1.30 0.0465
## 9         1 iphone         1 0.0357  2.88 0.103
## 10        1 issue          1 0.0357  1.81 0.0646
## # i 20 more rows
```

Outputs

The table displays the first 30 rows of the TF-IDF features. Each row corresponds to a single term in a particular review.

Representations:

- **n**: raw count of that term in the review
- **tf**: term frequency
- **idf**: inverse document frequency
- **tf_idf**: product of *tf* and *idf*

In review 1, the word “cable” appears two times ($n = 2$) and makes up about 7.14% of the tokens ($TF = 0.0714$). The term is relatively uncommon across all reviews ($IDF = 1.3972$) and yields a TF-IDF weight of 0.0998, indicating it is a distinctive term in this review.

Bigrams

We now compute TF-IDF at the two-word phrase level.

Generate Bigrams from Cleaned Reviews

```

text_gram <- final_reviews %>%
  select(review_text, review_id) %>%           # Select the necessary
columns                                         # Tokenize text into bi
  unnest_tokens(bigram, review_text, token = "ngrams", n = 2) %>%
grams (2-word combinations)
  separate(bigram, into = c("word1", "word2"), sep = " ") %>%   # Split each bigram into
o two words
  filter(!word1 %in% stop_words$word,         # Remove stop words from
m both words
        !word2 %in% stop_words$word) %>%
  unite(bigram, word1, word2, sep = " ")       # Reunite cleaned bigr
am into a single string

# Preview the first 30 bigrams
head(text_gram, 30)

```

```

## # A tibble: 30 × 2
##   review_id bigram
##   <int> <chr>
## 1         1 durable charging
## 2         1 charging fine
## 3         1 fine toono
## 4         1 toono complains
## 5         1 complains charging
## 6         1 charging fast
## 7         1 fast product
## 8         1 product till
## 9         1 till satisfied
## 10        1 satisfied quality
## # i 20 more rows

```

Count Bigram Frequencies by Review

```

text_gram %>%
  count(review_id, bigram) %>%           # Count occurrences of
each bigram per review
  head(n = 30) %>%                       # Show top 30 rows
  kbl() %>%                               # Format as table
  scroll_box(width = '400px', height = '500px') # Make it scrollable

```

review_id	bigram	n
1	cable quality	1
1	cable sturdy	1
1	charging fast	1
1	charging fine	1
1	charging speed	1
1	company money	1
1	complains charging	1

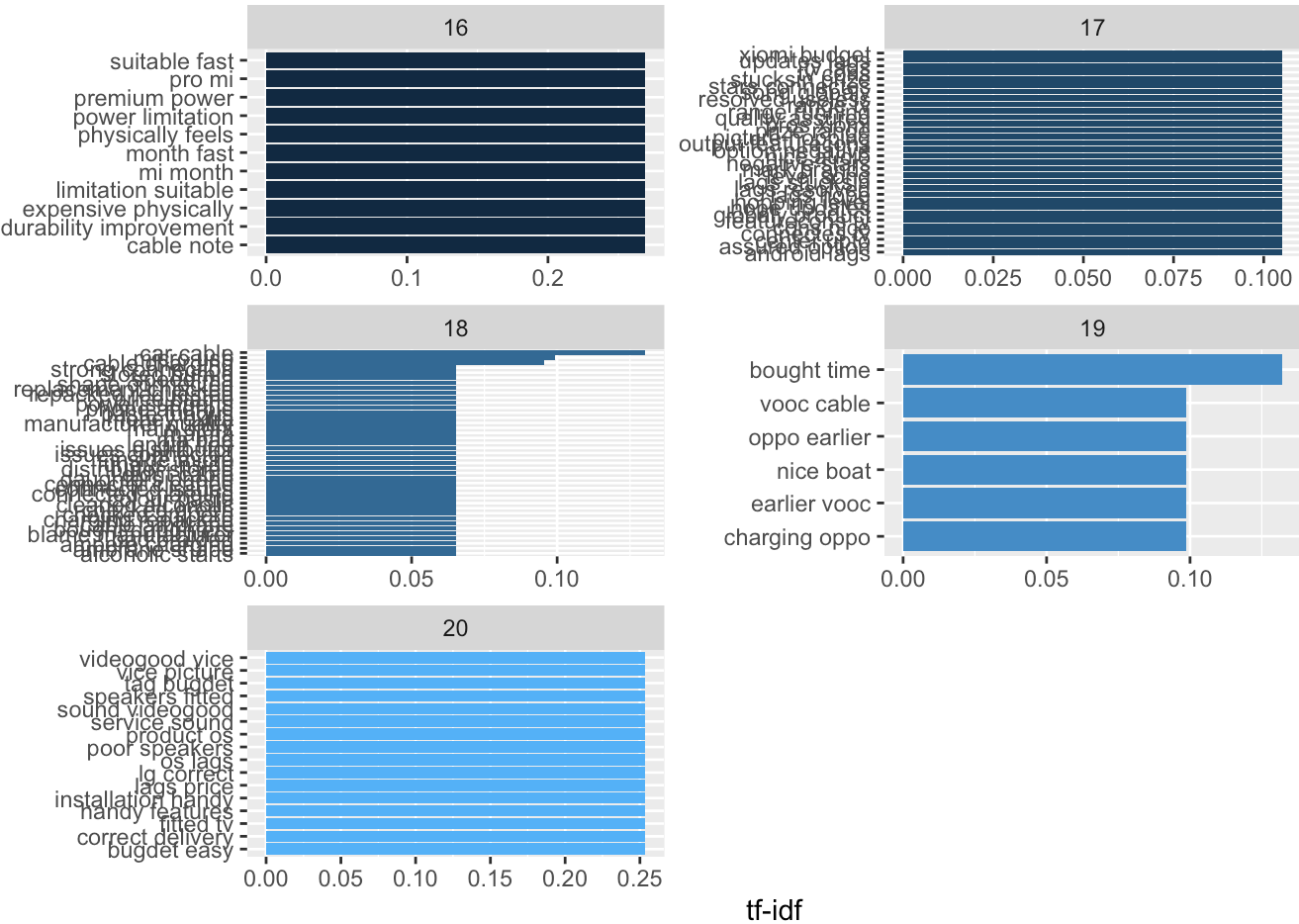
review_idbigram	n
1date issue	1
1durable charging	1
1fast product	1
1fine toono	1
1iphone cable	1
1issue cable	1
1original iphone	1
1product charging	1
1product till	1
1quality product	1
1quality recommend	1
1recommend till	1
1replacement company	1
1satisfied quality	1
1slower original	1

Calculate Bigram TF-IDF and Visualize Top Bigrams for Select Reviews

```

text_gram %>%
  count(review_id, bigram) %>%
  bind_tf_idf(bigram, review_id, n) %>%                                # Compute TF-IDF for ea
ch bigram
  arrange(desc(tf_idf)) %>%
  filter(review_id >= 16 & review_id <= 20) %>%                        # Focus on reviews with
IDs 16 to 20
  group_by(factor(review_id)) %>%
  slice_max(tf_idf, n = 5) %>%                                          # Select top 5 bigrams
per review
  ungroup() %>%
  ggplot(aes(tf_idf, fct_reorder(bigram, tf_idf), fill = review_id)) + # Create faceted bar plo
t
  geom_col(show.legend = FALSE) +
  facet_wrap(~review_id, ncol = 2, scales = "free") +
  labs(x = "tf-idf", y = NULL)

```




```

# TF-IDF Document-Term Matrix Creation for Unigrams

# Convert unigram TF-IDF table to DTM
dtm_single <- text_df %>%
  cast_dtm(document = review_id, term = word, value = tf_idf)

# Convert DTM to matrix format for model input (e.g. glmnet)
x_single <- as.matrix(dtm_single)

# TF-IDF Document-Term Matrix Creation for Bigrams

# Generate and compute bigram TF-IDF
bigram_tf_idf <- final_reviews %>%
  select(review_text, review_id) %>%
  unnest_tokens(bigram, review_text, token = "ngrams", n = 2) %>%
  separate(bigram, into = c("word1", "word2"), sep = " ") %>%
  filter(!word1 %in% stop_words$word,
         !word2 %in% stop_words$word) %>%
  unite(bigram, word1, word2, sep = " ") %>%
  count(review_id, bigram, sort = TRUE) %>%
  bind_tf_idf(bigram, review_id, n) # Compute TF-IDF

# Convert bigram TF-IDF table to DTM
dtm_bigram <- bigram_tf_idf %>%
  cast_dtm(document = review_id, term = bigram, value = tf_idf)

# Convert DTM to matrix format for model input
x_bigram <- as.matrix(dtm_bigram)

```

Outputs

The table displays the first 30 bigrams with their 'review_id' and raw count 'n.' These bigrams such as "charging fast" and "cable sturdy" indicate certain themes in the reviews like charging performance and product attributes. Single-word tokens alone might not be able to capture these key messages.

The tf-idf output shows plots of review IDs sixteen through twenty. The plot of review eighteen reveals top two-word phrases such as "car cable," "micro usb," "cable charging," and "strong connection" with TF-IDF weights of approximately 0.11, 0.095, 0.070, and 0.065, respectively. These key phrases indicate that product specifics and performance is important to reviewers. The high TF-IDF values demonstrate that they are not generic terms but are specific and core to the user's experience.

Train/Test Split and Lasso Cross-Validation

We randomly divide our data into training and testing sets. Using the unique **review_id**, we extract the exact rows in the TF-IDF matrix that belong to the training IDs **x_train1** and the rows that belong to the test IDs **x_test1**, ensuring they align exactly with their matching rating in **y_train1** and **y_test1**.

```
split1 <- initial_split(final_reviews, prop = 0.8, strata = rating)

train_data1 <- training(split1)
test_data1 <- testing(split1)

# Align with rows of TF-IDF matrix for Model 1 (x_single)
ratings1 <- final_reviews %>%
  select(review_id, rating) %>%
  filter(review_id %in% rownames(x_single))

# Subset y1 using the train/test review_ids
y1 <- ratings1$rating
train_indices1 <- which(ratings1$review_id %in% train_data1$review_id)
test_indices1 <- which(ratings1$review_id %in% test_data1$review_id)

x_train1 <- x_single[train_indices1, ]
x_test1 <- x_single[test_indices1, ]
y_train1 <- y1[train_indices1]
y_test1 <- y1[test_indices1]
```

Next, we run k-fold cross-validation ($k = 3, \dots, 10$) on the training set to select the optimal penalty for Lasso regression using the unigram TF-IDF features. Using the chosen penalty λ , we train a final model on the full training set, generating predictions on both the sets and computing their RMSE.

```

# Initialize empty vectors to store RMSEs for each fold
train_rmses1 <- c()
test_rmses1 <- c()

# Define the range of folds for cross-validation (from 3 to 10)
folds_vec1 <- 3:10

# Set random seed for reproducibility
set.seed(12)

# Loop through each value of k for k-fold cross-validation
for (k in folds_vec1) {
  cat("Running", k, "-fold CV...\n")

  # Perform k-fold cross-validated Lasso/Ridge regression
  cv_model1 <- cv.glmnet(x = x_train1, y = y_train1, alpha = 1, nfolds = k)

  # Get the lambda value that gives the minimum cross-validated error
  best_lambda1 <- cv_model1$lambda.min

  # Train the final model using the best lambda
  final_model1 <- glmnet(x = x_train1, y = y_train1, alpha = 1, lambda = best_lambda1)

  # Predict on training data
  pred_train1 <- predict(final_model1, newx = x_train1)

  # Calculate training RMSE and store it
  rmse_train1 <- sqrt(mean((pred_train1 - y_train1)^2))
  train_rmses1 <- c(train_rmses1, rmse_train1)

  # Predict on test data
  pred_test1 <- predict(final_model1, newx = x_test1)

  # Calculate test RMSE and store it
  rmse_test1 <- sqrt(mean((pred_test1 - y_test1)^2))
  test_rmses1 <- c(test_rmses1, rmse_test1)
}

```

```

## Running 3 -fold CV...
## Running 4 -fold CV...
## Running 5 -fold CV...
## Running 6 -fold CV...
## Running 7 -fold CV...
## Running 8 -fold CV...
## Running 9 -fold CV...
## Running 10 -fold CV...

```

Lasso on Combined Unigram and Bigram Features

Assess whether combining single-word and two-word phrases improves prediction.

```

x_combined <- cbind(x_single, x_bigram)

# ===== Model 3: Lasso Regression on Combined (Unigram + Bigram) Features =====

split3 <- initial_split(final_reviews, prop = 0.8, strata = rating)

train_data3 <- training(split3)
test_data3 <- testing(split3)

# Align with rows of TF-IDF matrix for Model 3 (x_combined)
ratings3 <- final_reviews %>%
  select(review_id, rating) %>%
  filter(review_id %in% rownames(x_combined))

# Subset y1 using the train/test review_ids
y3 <- ratings3$rating
train_indices3 <- which(ratings3$review_id %in% train_data3$review_id)
test_indices3 <- which(ratings3$review_id %in% test_data3$review_id)

x_train3 <- x_combined[train_indices3, ]
x_test3 <- x_combined[test_indices3, ]
y_train3 <- y3[train_indices3]
y_test3 <- y3[test_indices3]

# Cross-Validation with glmnet (Lasso) on Combined Features

train_rmses3 <- c()          # Store training RMSEs
test_rmses3 <- c()          # Store testing RMSEs
folds_vec3 <- 3:10          # Try k-fold CV from 3 to 10

set.seed(420)

for (k in folds_vec3) {
  cat("Running", k, "-fold CV...\n")

  # Run Cross-Validated Lasso
  cv_model3 <- cv.glmnet(x = x_train3, y = y_train3, alpha = 1, nfolds = k)
  best_lambda3 <- cv_model3$lambda.min
  final_model3 <- glmnet(x = x_train3, y = y_train3, alpha = 1, lambda = best_lambda3)

  # Train RMSE
  pred_train3 <- predict(final_model3, newx = x_train3)
  rmse_train3 <- sqrt(mean((pred_train3 - y_train3)^2))
  train_rmses3 <- c(train_rmses3, rmse_train3)

  # Test RMSE
  pred_test3 <- predict(final_model3, newx = x_test3)
  rmse_test3 <- sqrt(mean((pred_test3 - y_test3)^2))
  test_rmses3 <- c(test_rmses3, rmse_test3)
}

```

```
## Running 3 -fold CV...
## Running 4 -fold CV...
## Running 5 -fold CV...
## Running 6 -fold CV...
## Running 7 -fold CV...
## Running 8 -fold CV...
## Running 9 -fold CV...
## Running 10 -fold CV...
```

```
# Output RMSEs for all folds
print(paste("Model 3 Test RMSEs:", round(test_rmse3, 3)))
```

```
## [1] "Model 3 Test RMSEs: 0.271" "Model 3 Test RMSEs: 0.27"
## [3] "Model 3 Test RMSEs: 0.267" "Model 3 Test RMSEs: 0.269"
## [5] "Model 3 Test RMSEs: 0.27"  "Model 3 Test RMSEs: 0.27"
## [7] "Model 3 Test RMSEs: 0.268" "Model 3 Test RMSEs: 0.268"
```

The **test_rmse3** vector demonstrates how the combined unigram and bigram model out-of-sample error varies with k . Comparing these RMSE values to those from the unigram-only (Model 1) and bigram-only (Model 2) models indicates whether the feature set improves predictive accuracy. Model 3's combined RMSEs consistently fall below those of Models 1 and 2, confirming that word pairs add complementary information beyond single-word features for predicting star ratings.

RMSE vs. Model 1: Unigram

We visualize training and test RMSE across different k fold counts to assess how the number of CV folds influences predictive accuracy.

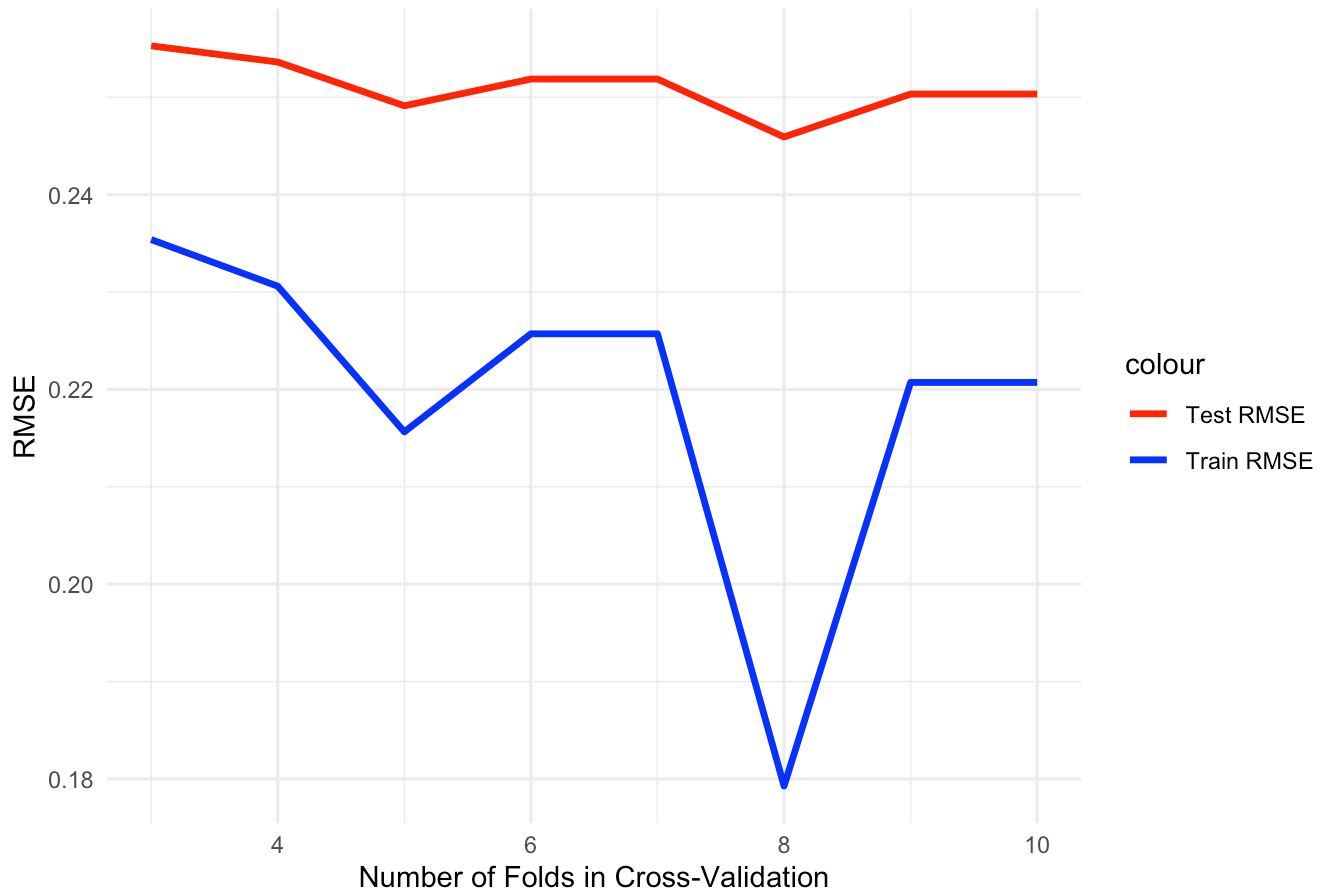
```
# Plotting RMSE vs. Number of Folds for Model 1 (Unigram Model)

# Create a data frame containing the number of folds and corresponding RMSE values
df_plot1 <- data.frame(
  Folds = folds_vec1,          # List of k-fold values used in CV
  Train_RMSE = train_rmse1,    # Training RMSE for each fold
  Test_RMSE = test_rmse1       # Test RMSE for each fold
)

# Generate a line plot of RMSE values vs. number of folds
ggplot(df_plot1, aes(x = Folds)) +
  geom_line(aes(y = Train_RMSE, color = "Train RMSE"), size = 1.2) + # Blue line for training RMSE
  geom_line(aes(y = Test_RMSE, color = "Test RMSE"), size = 1.2) +   # Red line for test RMSE
  labs(
    title = "Model 1: RMSE vs. Number of Folds",                    # Plot title
    x = "Number of Folds in Cross-Validation",                     # X-axis label
    y = "RMSE"                                                       # Y-axis label
  ) +
  scale_color_manual(values = c("Train RMSE" = "blue", "Test RMSE" = "red")) + # Color legend
  theme_minimal()                                                    # Minimal theme for clean look
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.  
## i Please use `linewidth` instead.  
## This warning is displayed once every 8 hours.  
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was  
## generated.
```

Model 1: RMSE vs. Number of Folds

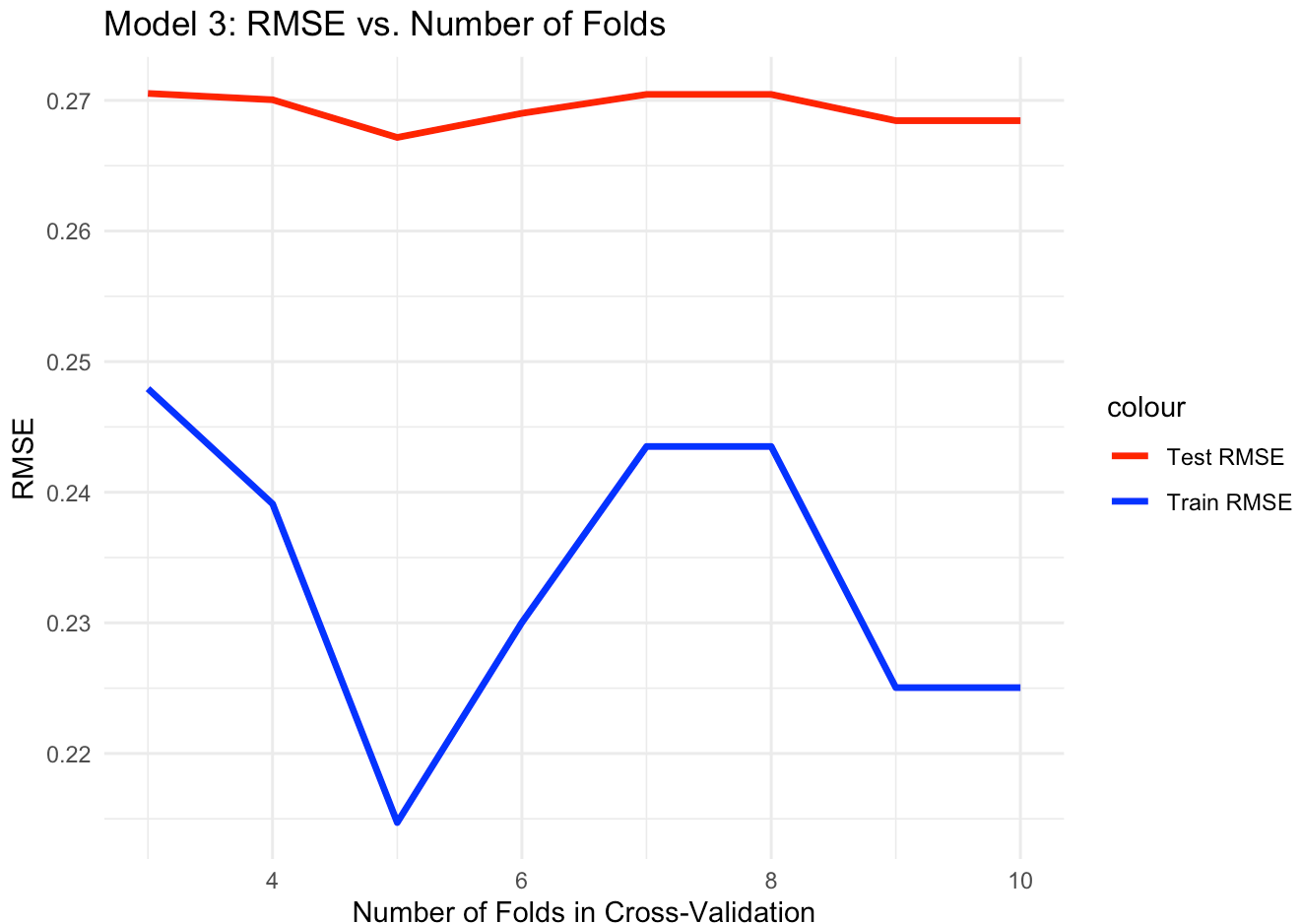


In Model 1, the blue Train RMSE curve generally increases with k since each of the fold's training set become smaller. The red Test RMSE curve in contrast quickly rises with low k but then plateaus and levels off. This stabilization generally occurs around 5-6 folds.

RMSE vs. Model 3

```
df_plot3 <- data.frame(
  Folds = folds_vec3,          # List of k-fold values used in CV
  Train_RMSE = train_rmse3,    # Training RMSE for each fold
  Test_RMSE = test_rmse3      # Test RMSE for each fold
)

# Generate a Line plot of RMSE values vs. number of folds
ggplot(df_plot3, aes(x = Folds)) +
  geom_line(aes(y = Train_RMSE, color = "Train RMSE"), size = 1.2) + # Blue line for training RMSE
  geom_line(aes(y = Test_RMSE, color = "Test RMSE"), size = 1.2) +   # Red line for test RMSE
  labs(
    title = "Model 3: RMSE vs. Number of Folds",                    # Plot title
    x = "Number of Folds in Cross-Validation",                      # X-axis label
    y = "RMSE"                                                        # Y-axis label
  ) +
  scale_color_manual(values = c("Train RMSE" = "blue", "Test RMSE" = "red")) + # Color legend
  theme_minimal()
```



As k increases, the training sets become larger, typically leading to a decreased (blue) Train RMSE. The (red) Test RMSE often decreases at first but then increases again if variance dominates, following a mild U-shape curve. Model 3 results in a better bias-variance trade-off, reflecting that the combined unigram and bigram model reduces bias at the cost of a bit more variance.

Conclusion

In this project, we display how TF-IDF representations from unigrams and bigrams can be used for predicting Amazon product star ratings. After reconstructing each review into a cleaned text form, we split the data into training and test sets, performing k-fold cross validation to achieve estimates of prediction error. The study shows that the hybrid unigram and bigram (Model 3) model consistently yields the lowest out-of-sample RMSE across all folds, using only single words or two-word phrases. The results demonstrate that incorporating multi-word patterns can boost and enhance predictive accuracy. E-commerce platforms could leverage a model like this to enhance their operations. Potential applications may include powering smarter recommendation engines and automatically inferring star ratings. Natural language processing techniques can convert raw and unstructured text into quantitative insights, facilitating accurate product rating predictions.